

CS50 Cybersecurity

Lectures 2, 3 & 4 · Securing Systems + Securing Software + Preserving Privacy

LECTURE 2

Securing Systems

The Big Picture

This lecture shifts from encryption theory to networks and the real systems that carry your data. The core question: once data leaves your device, how is it protected — and where does it break down?

Wi-Fi — Encrypted vs Unencrypted Networks

- Unsecured networks send your data in plain text — anyone nearby can read it
- Secured networks use encryption to protect data between you and other devices
- WPA (Wi-Fi Protected Access) is the encryption standard used to secure wireless networks

Easy way to remember it

An unsecured Wi-Fi is like shouting your messages across a crowded room. WPA is like whispering through a sealed envelope — others can see you talking, but can't read the message.

HTTP — The Unencrypted Protocol

- HTTP (Hypertext Transfer Protocol) transfers data with zero encryption — everything is readable
- Man-in-the-Middle (MitM) attacks: an adversary intercepts traffic and can inject malicious HTML or advertisements into every page you visit
- Packet sniffing: adversaries can capture and read raw data in transit — including credit card numbers
- Session hijacking: cookies (small files websites store on your computer) can be stolen. An adversary injects a 'supercookie' to track and impersonate you

HTTPS — Encrypted HTTP

- HTTPS encrypts all traffic between you and a website using TLS (Transport Layer Security)
- Uses public key cryptography: the website has a public key + private key
- Certificate Authorities (CAs) are trusted companies that sign website certificates (X.509 format)

How your browser verifies a website:

1. Browser downloads the website's certificate
2. Runs it through a hash algorithm to produce a hash
3. Uses the website's public key + certificate signature to verify that hash
4. If the two hashes match → the connection is secure and the site is legitimate

→ SSL Stripping attack: adversary downgrades your HTTPS connection to HTTP, then redirects you to a malicious site

→ HSTS (HTTP Strict Transport Security) defends against this — the server tells your browser to always use a secure connection, no exceptions

VPN — Virtual Private Network

→ Creates a fully encrypted tunnel between your device and a second server

→ All traffic flows through that encrypted tunnel — nothing is visible to outsiders

→ Your visible IP address becomes the VPN server's IP, not your own

→ Common use: masking your location to appear as if you're browsing from another country

Easy way to remember it

A VPN is like mailing your letter inside a second sealed envelope addressed to a trusted friend. They open the outer envelope and forward your letter — the post office only ever sees the outer address.

SSH — Secure Shell

→ SSH lets you securely execute commands on a remote server over an encrypted connection

→ Requires proper credentials and permissions — SSH alone doesn't grant access

Example SSH command:

`ssh stanford.edu` → connects to a server at Stanford (if you have access)

Ports — Traffic Directors

→ Port numbers tell a server which service incoming traffic is meant for

Port 80	HTTP — unencrypted web traffic
Port 443	HTTPS — encrypted web traffic
Port 22	SSH — secure remote access

→ Port scanning: adversaries probe all ports to find which ones accept traffic

→ Penetration testing: professionals do this legally to find vulnerabilities before attackers do

→ Ethical hacking: the legal business of discovering and reporting security weaknesses

- Firewalls: software that blocks unauthorized access to ports using IP address rules
- Deep packet inspection: firewalls can look inside data packets — used by schools/companies to monitor or block certain traffic via a proxy in the middle

Malware — Malicious Software

Virus	Attaches to your computer and can do almost anything once installed
Worm	Spreads automatically between computers by exploiting security holes — no user action needed
Botnet	Infects your machine and thousands of others; attacker controls all of them remotely
DDoS	Botnet computers flood a server with requests to slow or shut it down. 'Distributed' because it comes from thousands of different IP addresses simultaneously

Antivirus Software

- Detects known viruses and attempts to remove them
- Must be kept updated — without updates, new threats slip through
- Zero-day attacks exploit unknown vulnerabilities before the software company can patch them — antivirus cannot protect against what it doesn't know yet

Quick-Reference Summary

- ★ WPA encrypts wireless networks — always prefer secured networks
- ★ HTTP is unencrypted — vulnerable to MitM attacks, packet sniffing, and session hijacking
- ★ HTTPS uses TLS + public key cryptography — CAs verify legitimacy via certificate hashes
- ★ HSTS forces HTTPS — defends against SSL stripping attacks
- ★ VPN = encrypted tunnel that hides your traffic and masks your IP address
- ★ SSH = secure remote command execution — encrypted, requires credentials
- ★ Ports direct traffic: 80 (HTTP), 443 (HTTPS), 22 (SSH)
- ★ Port scanning = adversaries probe for open ports. Firewalls block unauthorized access
- ★ Viruses attach, worms spread, botnets coordinate, DDoS overwhelm
- ★ Antivirus can't stop zero-day attacks — unknown exploits have no patch yet

LECTURE 3

Securing Software

The Big Picture

This lecture focuses on how adversaries exploit software — and how developers defend against it. The recurring rule: never trust user input, ever.

Phishing — Fake Links and Fake Pages

- Phishing tricks you into providing sensitive information by disguising malicious destinations
- HTML anchor tags display one URL but link to another:

`https://harvard.edu`

- The user sees 'harvard.edu' but clicks through to yale.edu
- Adversaries clone real websites to harvest your credentials

- Defence: hover over links before clicking. Check the actual URL in your browser's address bar

Code Injection / Cross-Site Scripting (XSS)

- XSS: an attacker inserts malicious code into a website's input fields — the site then blindly runs it
- If a site outputs user input without checking it, that input can be a script:

`alert('attack')` → inserted into a search box, this could run in the browser

- Reflected attack: attacker crafts a link containing malicious code in the URL. When the victim clicks it, their own browser executes the attack against themselves
- Stored attack: malicious code is saved on the server (e.g. in an email or comment). Every user who loads that content becomes a victim

Defences Against XSS — Character Escapes & HTTP Headers

- Character escaping converts dangerous characters into harmless text representations

<	Displays a less-than sign < without triggering a tag
>	Displays a greater-than sign >
&	Displays an ampersand &
"	Displays a double quote "

'	Displays a single quote '
---	---------------------------

→ Content-Security-Policy HTTP header: tells the browser to only load scripts from approved external .js files — inline script tags are blocked entirely

SQL Injection — Attacking the Database

SQL is the language used to query databases. If user input is inserted directly into SQL queries without sanitisation, an attacker can rewrite the query.

Normal query: `SELECT * FROM users WHERE username = '{username}'`
 Attacker input: `malan'; DELETE FROM users; --`
 What runs: `SELECT * FROM users WHERE username = 'malan'; DELETE FROM users; --'`
 → Every account in the database is deleted

Password bypass — attacker enters: `' OR '1'='1`
 What runs: `WHERE (username = 'malan' AND password = ") OR '1'='1'`
 → `'1'='1'` is always true → query returns all users → attacker gains access

→ Prepared statements are the fix — the query template is defined first, then user input is inserted safely. Single quotes are automatically escaped so code cannot be injected

Easy way to remember it

A prepared statement is like a form with locked fields. You can fill in the blanks, but you can't rewrite the form itself.

Command Injection

- If user input is passed directly to the command line (via `system()` or `eval()`), an attacker can execute any command on the server
- Always sanitise and escape input before passing it anywhere near a shell
- Read the documentation for your language/framework — it specifies the safe way to handle this

Developer Tools & Client-Side Validation

- HTML lives on the user's machine. Using browser developer tools, anyone can edit it locally
- A 'disabled' checkbox or a 'required' field can be removed in seconds via dev tools
- This means client-side validation (validation done in the browser) is never sufficient alone

Golden rule: Always validate on the server-side. Never trust what the browser sends you.

Cross-Site Request Forgery (CSRF)

- CSRF tricks a victim's browser into sending an authorised request to a website without the victim's knowledge
- GET method risk: an image tag can silently trigger a GET request (e.g. automatically purchasing a product)
- POST method risk: a hidden form on an attacker's site can auto-submit and make transactions on another site using the victim's active session

Defence — CSRF tokens:

- The server generates a unique secret token per user per session
- Every form submission must include this token
- An attacker's fake form won't have the correct token → request is rejected

Arbitrary Code Execution (ACE)

- ACE = running unintended code within a program
- Buffer overflow: program receives more input than its memory can handle — overflow spills into adjacent memory and can crash or hijack the program
- Stack overflow: a variant where the call stack is overflowed to inject and execute malicious code
- These techniques are also used for software cracking (bypassing payment) and reverse engineering

Software Distribution & Trust

Open-source	Code is public — anyone can audit it. More eyes = faster vulnerability discovery. But attackers can also read the code.
Closed-source	Code is private — harder for attackers to study. But fewer eyes means some vulnerabilities go unnoticed longer.
App stores	Google and Apple sign all submitted apps with digital signatures. Operating systems only install verified, signed software — raising the bar for malicious code.
Package managers	Use a similar signing mechanism for developer libraries. Not foolproof, but significantly safer than downloading from random sources.

Finding & Tracking Vulnerabilities

- Bug bounties: companies pay researchers to find and report vulnerabilities — financially incentivising attackers to become defenders

CVE	Common Vulnerabilities and Exposures — a database of known vulnerabilities worldwide
------------	--

CVSS	Common Vulnerability Scoring System — rates how severe each vulnerability is
EPSS	Exploit Prediction Scoring System — predicts the likelihood of a vulnerability being exploited
KEV	Known Exploited Vulnerabilities database — a list of vulnerabilities actively being used by attackers

Quick-Reference Summary

- ★ Phishing uses deceptive links and cloned websites — always check the real URL
- ★ XSS = malicious scripts injected via user input — reflected (via URL) or stored (on server)
- ★ Character escaping neutralises dangerous characters. CSP headers block inline scripts
- ★ SQL injection rewrites database queries — use prepared statements, always
- ★ Command injection executes shell commands from user input — sanitise everything
- ★ Client-side validation is trivially bypassed — server-side validation is mandatory
- ★ CSRF tricks your browser into making unauthorised requests — CSRF tokens prevent this
- ★ Buffer/stack overflows can execute arbitrary code — validate input length
- ★ App stores + digital signatures raise the bar for malicious code installation
- ★ CVE, CVSS, EPSS, KEV — the tools professionals use to track and prioritise threats

LECTURE 4

Preserving Privacy

The Big Picture

Even when you're not being hacked, you are being tracked. This lecture is about all the ways your data is collected without your direct knowledge — and what you can actually do about it.

Web Browsing History — Not As Private As You Think

- Clearing your browser history only deletes your local copy
- Servers keep logs of every request they receive: your IP address, timestamp, what you accessed
- Your ISP, the DNS system, and every server you contact all have their own records

HTTP Headers — You're Sharing More Than You Know

- When you click a link, your browser automatically sends a Referer header — telling the destination site which page sent you there

Example: you search Google for 'cats' then click a result
The destination site receives: Referer: <https://www.google.com/search?q=cats>
→ The site now knows exactly what you searched for

- Websites can add a meta tag to limit what they share:

	Sends only the domain — not the full URL or search query
	Sends nothing — maximum privacy

Fingerprinting — Being Identified Without Cookies

- Even without cookies, servers can identify you from the combination of details your browser shares

User-Agent header	Your browser type, OS version, device model
IP address	Your approximate location, logged by every server you visit
Screen resolution	Unique enough to narrow down your device
Installed fonts/extensions	Surprisingly identifying — most users have a unique combination

- Over time, combining these creates a unique fingerprint — even private browsing doesn't help

Cookies — The Three Types You Should Know

Session cookies	Track your current session (e.g. keeping you logged in). Expire when you close the browser or after a set time.
Tracking cookies	Set by third parties to follow you across the web. A Google Analytics cookie can last 2 years and reports your activity on every site that uses it.
Third-party cookies	Loaded by ads or embedded content from other domains. If you visit harvard.edu and yale.edu and both load the same ad from example.com, example.com now knows you visited both sites.

Tracking Parameters — Visible in Plain Sight

→ Unlike cookies (hidden in the background), tracking parameters are right there in the URL

Tracked URL: `https://example.com/ad?click_id=YmVhODI1MmZmNGU4&campaign_id=23`

→ `click_id` uniquely identifies you

Sanitised URL: `https://example.com/ad?campaign_id=23`

→ Modern browsers increasingly strip individual tracking IDs from URLs

Private Browsing — What It Actually Does (and Doesn't)

→ Clears past cookies from the session — old tracking data doesn't carry over

→ New cookies can still be set within the private session — you're not invisible

→ Servers can still log your IP, fingerprint your browser, and track your activity

→ Your ISP still sees all your DNS requests and traffic patterns

Easy way to remember it

Private browsing is like using a borrowed library card instead of your own. The librarian still sees you, still logs what you borrow — you just don't have your name on the card.

Supercookies — ISP-Level Tracking

→ Your Internet Service Provider (ISP) can inject tracking identifiers directly into your HTTP headers — without your knowledge or consent

→ These bypass browser cookie controls entirely

→ Some ISPs offer an opt-out — check your provider's privacy settings

DNS — The Unencrypted Address Book

→ DNS (Domain Name System) translates domain names (harvard.edu) into IP addresses

→ By default, DNS traffic is completely unencrypted — your ISP and DNS provider see every site you attempt to visit

DoH — DNS over HTTPS	Encrypts your DNS requests inside HTTPS — hides them from your ISP
DoT — DNS over TLS	Same goal, different protocol — also encrypts DNS lookups

VPN — Privacy Limitations

- VPN creates an encrypted tunnel: your traffic appears to come from the VPN server, not you
- Hides your activity from your ISP and most observers on the network
- Does NOT protect you if your device is infected with malware — malware operates inside your machine, before the VPN tunnel
- You are trusting the VPN provider — they can still see your traffic

Tor — Maximum Anonymity (With Caveats)

- Tor routes your traffic through multiple encrypted nodes — each node only knows the previous and next hop, never the full picture
- Designed to retain minimal logs of your activity
- Provides very high anonymity — but not absolute

Important caveat: if you are the only person using Tor on your school or work network, network administrators may be able to identify you through other means — even if they can't read your traffic.

- No technology provides absolute protection — it's always about raising the cost for adversaries

App Permissions — What You're Granting

- Operating systems increasingly ask apps for explicit permission before accessing your location, camera, microphone, contacts, etc.
- Maps apps (Apple Maps, Google Maps) know your precise location at all times if granted permission
- Review app permissions regularly — revoke access you no longer need

Quick-Reference Summary

- ★ Clearing browser history only deletes your local copy — servers keep their own logs
- ★ The Referer header tells sites where you came from — including your search query
- ★ Fingerprinting identifies you from browser/device details — even without cookies
- ★ Session cookies track logins. Tracking cookies follow you for years across sites

- ★ Third-party cookies let ad networks map your journey across unrelated websites
- ★ Tracking parameters in URLs identify you — modern browsers are starting to strip them
- ★ Private browsing clears old cookies but doesn't hide you from servers or your ISP
- ★ Supercookies are injected by ISPs into your HTTP headers — check for opt-out options
- ★ Default DNS is unencrypted. Use DoH or DoT to encrypt your DNS lookups
- ★ VPN hides traffic from ISP but doesn't protect against malware on your own device
- ★ Tor routes through multiple encrypted nodes — high anonymity but not absolute
- ★ Review app permissions regularly — location access in particular is very revealing

How Lectures 2, 3 & 4 Connect to Lectures 0 & 1

Encryption (L1)	Powers HTTPS, VPNs, SSH, and WPA. The same maths from Lecture 1 is what makes all of Lecture 2's 'secure' protocols actually secure.
Phishing (L0)	Introduced in Lecture 0 as a social engineering attack. Lecture 3 shows the technical mechanism: forged anchor tags and fake login pages.
Passwords (L0)	SQL injection can bypass password checks entirely — '1'=1' renders the password field meaningless. Good password storage (L1) doesn't help if the login logic is broken.
HTTPS (L2)	Lecture 3's CSRF attacks show that HTTPS alone isn't enough — the connection is secure, but the request can still be forged.
Cookies (L2)	Lecture 4 expands on this: tracking cookies, third-party cookies, and supercookies are all extensions of the session cookie concept.
VPN (L2 + L4)	Lecture 2 introduces VPNs as a security tool. Lecture 4 clarifies their privacy limitations — they hide traffic but don't stop malware and require trusting the VPN provider.
Digital signatures (L1)	The same public/private key system that secures passkeys also powers app store code signing — ensuring only authorised software is installed.

The thread across all four lectures:

Security is about raising the cost for adversaries — in time, computing power, and knowledge — until the attack isn't worth attempting. Privacy is about limiting what you reveal, even to parties you nominally trust.